Outcome 13 Rubric

std	P (Proficient)	S (Sufficient)	D (Developing)	I (Insufficient)	N (No evidence)
13.1	Designs and implements classes that utilize custom comparisons such as __eq__() or __lt__() AND demonstrates how to invoke comparisons with 2 or more instances of the object AND explains the design with comments and appropriate naming conventions.	Designs and implements classes that utilize custom comparisons such as __eq__() or __lt__() AND demonstrates how to invoke comparisons with 2 or more instances of the object.	Minor syntax issues in class design OR does not compare instances of the objects but the class is implemented correctly.	Incorrect logic behind comparisons (eg. comparing wrong components of objects).	No evidence method overriding.
13.2	Correctly overrides built-in behavior of methods such as __and__() or __mul__() appropriately for the context of the problem AND either (demonstrates how to combine 2 or more instances of the object OR modifies a single instance) AND explains the design with comments and appropriate naming conventions.	Correctly overrides built-in behavior of methods such as __and__() or __mul__() appropriately for the context of the problem AND either (demonstrates how to combine 2 or more instances of the object OR modifies a single instance)	Minor syntax issues in class design OR does not combine/modify instances of the object(s) but the class is implemented correctly.	Incorrect logic behind combining/ modifying object (eg. combining/ modifying wrong component(s) of objects).	No evidence method overriding.
13.3	Correctly overrides built-in behavior of methods such as __and__() to modify an objects internal list or dictionary AND prints the result (using __str__) of combining at least 2 instance of the object AND explains the design with comments and appropriate naming conventions.	Correctly overrides built-in behavior of methods such as __and__() to modify an objects internal list or dictionary AND prints the result (using __str__) of combining at least 2 instance of the object.	Minor syntax issues in class design OR does not combine instances of the object(s) but the class is implemented correctly.	Incorrect class structure or major misunder-standings of how class components work together.	No functional class structure provided.